Golden Spoon Cuisine — 3-Day Build Plan

Nov 6, 2026 · @Chike

Overview

Three days, about 34 focused hours, to deploy the Golden Spoon Cuisine ordering, rewards, events and admin app on AWS. The days follow the brief's priority order: ordering first, then service enquiries, then admin. If time runs out, what is finished still works as a product.

Day	Focus	Hours	Checkpoint
1	Backend foundations and ordering engine	~11	Journeys 1 and 2 work through the API; unit tests pass
2	Customer app	~11	Journeys 1 and 2 work in a 360px phone view
3	Events, admin, deploy	~12	All 4 journeys work on a live CloudFront URL

Stack: TypeScript, React 19 + Vite, TanStack Query, Zustand, Tailwind v4, Node 24 + Express 5, Zod, Drizzle, PostgreSQL 17, Docker Compose, Terraform, AWS (ECR, ECS Fargate, RDS, ALB, S3, CloudFront, Secrets Manager), GitHub Actions.

The four journeys (from the brief):

1. Browse a dish → customise → add to cart → checkout → simulated payment → order confirmation.
2. View loyalty progress → redeem an available reward.
3. Browse event services → request a quote → view an example quote → accept.
4. Open an order in the admin dashboard → update its status.

Before Day 1

Do these before the clock starts, so Day 1 opens with code rather than installs.

- [ ] Install Node 24 LTS, pnpm, Docker Desktop, Git, the Terraform CLI and the AWS CLI
- [ ] Create a GitHub repo `golden-spoon`
- [ ] Create an AWS account, an admin IAM user for your laptop, and a billing alarm at about $10

- [ ] Install an HTTP client for API testing (Bruno, or the VS Code REST Client for `.http` files)
- [ ] Ask the client for real Nigerian food and event photos; use clearly labelled placeholders until they arrive
- [ ] Write down the seed data: about 25 menu items with prices, the Awka delivery areas and fees, and opening hours per branch

Day 1 — Backend foundations and ordering engine (~11h)

By tonight the API takes an order from cart to paid and confirmed, and runs the loyalty rules, all against real Postgres.

#	Block	Hours	What you build	Main skills
1	Setup	1	pnpm monorepo (`apps/web` , `apps/api` , `packages/shared`), strict TypeScript, ESLint, Prettier, Docker Compose with Postgres, MinIO and Adminer, environment config validated with Zod	Monorepo tooling, Docker Compose
2	Schema, migrations, seed	2.5	Every table, including the events tables: branches, zones, opening hours, settings, categories, menu items, option groups, options, users, refresh tokens, addresses, orders, order items, order item options, order status events, payments, loyalty ledger, rewards, services, enquiries, quotes, quote line items. Seed the menu, zones, services and a demo admin	Postgres modelling, Drizzle, SQL migrations

#	Block	Hours	What you build	Main skills
3	Domain logic and unit tests	2	Pure functions in `domain/`: pricing in kobo, option validation (required, min, max), the order state machine, loyalty earn, revoke and redeem, Nigerian phone normalisation to +234, naira formatting	TypeScript types, Vitest, TDD
4	API	4.5	Routes for the menu, zones and hours, `POST /cart/quote`, order creation with an idempotency key, simulated card and bank-transfer payments, order tracking, auth (register, login, refresh, me), addresses, loyalty status and redeem. Error middleware with RFC 7807 problem details, pino logging	Express 5, layering (routes → service → repo), JWT, Argon2
5	Containerise the API	1	Multi-stage Dockerfile; the API runs in Compose alongside Postgres	Docker images, multi-stage builds

Checkpoint at the end of Day 1:

- [ ] Journey 1 works end to end in `.http` files: quote, order, pay, confirmed
- [ ] Journey 2 works: complete 5 qualifying orders, the reward appears, redeem it on a 6th order
- [ ] Unit tests pass for pricing, the state machine and loyalty
- [ ] A failed payment, and an order outside the delivery areas, both return clear errors

Day 2 — Customer app (~11h)

By tonight a customer on a small phone can order, pay (demo) and track a meal, create an account, and redeem a reward.

#	Block	Hours	What you build	Main skills
1	Design system and app shell	2	Tailwind tokens for the brand palette, Playfair Display and Inter. Button, Field, Badge, Dialog, Tabs, Toast, FoodCard, Skeleton, EmptyState, StampCard. Charcoal header, bottom nav (Home, Menu, Orders, Account), floating cart button that never overlaps controls	Tailwind v4, Radix primitives, accessible components
2	Home, menu, dish detail	3	Home with location selector, hero, "Order food" and "Plan an event" actions, popular dishes, reward preview, service cards, gallery, WhatsApp link. Menu with search and category chips, sold-out states. Customisation sheet with live price, included vs paid breakdown, inline validation	React Router, TanStack Query, React Hook Form + Zod
3	Cart, checkout, payment, confirmation	3.5	Cart persisted in Zustand with fees and total from `/cart/quote`. Delivery or pickup, area and landmark, outside-coverage message. Demo payment with pending, success and failed states. Order number and status timeline that polls every 10 seconds	Client vs server state, optimistic UI, polling

#	Block	Hours	What you build	Main skills
4	Auth, account, rewards	2.5	Sign up and log in with the refresh-token flow. Order history with reorder, saved addresses, enquiries tab (empty for now). Rewards screen with five stamps, rules and a redeem action	Auth on the client, protected routes, empty states

Checkpoint at the end of Day 2:

- [ ] Journey 1 works in the browser at a 360px width
- [ ] Journey 2 works in the browser
- [ ] Loading, empty, error and success states show on the menu, cart and orders screens
- [ ] Images are lazy-loaded and sized; the admin code is not in the customer bundle

Day 3 — Events, admin, deploy (~12h)

By tonight all four journeys work on a live CloudFront URL, and pushing to `main` deploys.

#	Block	Hours	What you build	Main skills
1	Events and quotes	3	API for services, enquiries, quotes, accept and demo deposit, with the enquiry state machine. UI: service catalogue and detail pages, a 4-step enquiry form, "My enquiries", quote view with line items, deposit and an Accept button. Copy says "requests a quotation", never "booked"	Multi-step forms, a second state machine

#	Block	Hours	What you build	Main skills
2	Admin dashboard	4	Admin-only routes (role middleware). Sidebar on desktop, drawer on mobile. Orders board with next-status buttons, menu price and availability editor, enquiries and quote builder, booking list or calendar, settings for loyalty, zones, fees and hours, sales overview from SQL aggregates on demo data	Role-based access, SQL aggregates, responsive layout
3	Deploy	4	Terraform with S3 remote state: VPC, RDS, ECR, ECS Fargate + ALB, S3 + CloudFront (`/api/*` to the ALB), Secrets Manager. First deploy from your laptop, then GitHub Actions with OIDC. Playwright smoke test of the 4 journeys on the live URL	Terraform modules, AWS networking, CI/CD
4	Buffer	1	Fix whatever broke; run `terraform destroy` when not demoing	Debugging

Checkpoint at the end of Day 3:

- [ ] Journey 3 works: enquiry, quote, accept, demo deposit, confirmed
- [ ] Journey 4 works: admin moves an order to Preparing and the customer timeline updates
- [ ] The app is live on the CloudFront URL
- [ ] A push to `main` runs tests and deploys

If you fall behind

Cut from the top of this list first:

1. Show bookings as a list grouped by date instead of a calendar grid.
2. Use seeded image URLs instead of uploading to S3 from admin.

3. Use a single JWT instead of refresh-token rotation.
4. Show stat tiles only, with no sales chart.
5. Deploy with a manual `terraform apply` and Docker push instead of the CI/CD pipeline.

Never cut: server-side pricing, the two state machines, idempotency on orders and payments, and the append-only loyalty ledger. These are what make this real engineering rather than a demo.

Biggest risk: if Terraform and AWS are new to you, Day 3's deploy block is where things go wrong. A safer version is to spend Day 1's last hour setting up Terraform remote state and an ECR repo, so the cloud setup isn't brand new on the final afternoon.

Cost: the ALB and RDS cost money while running. Keep the billing alarm on and run `terraform destroy` between demos.